

Capitolo 7 - Programmi e Programmazione

Terminologia della Qualità e Problematiche di Base

Per comprendere i problemi di sicurezza nei programmi e nella programmazione, è fondamentale distinguere la terminologia legata alla mancanza di qualità del software.

In passato gli ingegneri della sicurezza non facevano molta distinzione, ma oggi si identificano tre concetti chiave:

- **Error (Errore):** uno sbaglio commesso da un essere umano.
- **Fault (Difetto/Anomalia):** un passaggio, un comando, un processo o una definizione di dati errati all'interno di un programma, di un progetto o della documentazione.
- **Failure (Guasto/Fallimento):** una deviazione del sistema rispetto al suo comportamento richiesto.

È importante notare che i documenti dei requisiti possono contenere dei fault; pertanto, un failure indica che il sistema non sta funzionando come richiesto, anche se potrebbe funzionare esattamente come specificato nei requisiti fallati.

Sviste di Programmazione

Alla base di molti difetti troviamo le sviste di programmazione (**oversights**), ovvero fallimenti involontari nel notare o fare qualcosa durante la stesura del codice. I difetti di un programma (**flaws**) hanno due implicazioni di sicurezza principali:

- possono causare problemi di integrità (portando a output o azioni dannose)
- possono offrire l'opportunità di essere sfruttati da un attore malintenzionato.

Un'operazione incorretta rappresenta una violazione dell'integrità, la quale non riguarda solo la correttezza, ma anche l'accuratezza, la precisione e la coerenza. Un programma difettoso può infatti modificare in modo inappropriato dati precedentemente corretti, talvolta *sovrascrivendoli* o cancellandoli. In sintesi, anche un difetto di natura benigna può trasformarsi in una vulnerabilità se un utente malintenzionato scopre come manipolarlo a proprio vantaggio. Alcuni esempi di *programming flaws* sono:

1. Buffer Overflow
2. Incomplete mediation
3. Time-of-Check to Time-of-Use
4. Race condition
5. ...

Buffer Overflow

Il **Buffer Overflow** (Traboccamento del buffer) si verifica quando un programma supera lo spazio di memoria allocato per il buffer.

Individuare e prevenire questi difetti è molto difficile e l'impatto di un overflow può essere subdolo e sproporzionato rispetto alla svista iniziale. Spesso, il buffer overflow rappresenta il primo passo per lanciare un attacco molto più dannoso, come l'apertura di una shell con privilegi elevati. Un esempio storico è il caso di "**Dialer.exe**" scoperto da Litchfield nel 1999: l'inserimento di un numero di telefono di 100 cifre causava il crash del programma perché il programmatore non aveva imposto un limite superiore all'input. Litchfield scoprì che controllando i dati che sovrascrivevano lo stack, poteva reindirizzare l'esecuzione del programma verso qualsiasi istruzione desiderata.

In memoria, i dati e il codice sono indistinguibili tra loro, essendo entrambi sequenze di bit, e l'origine del codice non è visibile al processore. Un utente malintenzionato sfrutta questo principio inserendo valori specifici nei dati in modo che trabocchino nel codice eseguibile, facendoli interpretare come istruzioni valide. L'attacco prende di mira la struttura delle chiamate procedurali (*Procedure call*). Normalmente, quando viene chiamata una procedura, parametri, indirizzo di ritorno (*program counter*) e vecchio *stack pointer* vengono salvati nello Stack. L'attaccante inietta i propri dati per sovrascrivere l'indirizzo di ritorno; così facendo, quando la sottoprocedura termina, il controllo viene trasferito al codice malevolo scritto dall'attaccante. Questo tipo di overflow può avvenire anche tramite lo *Heap Smashing*, dove i dati dell'Heap (la cui dimensione cresce dinamicamente) si espandono fino a collidere con lo Stack.

La sovrascrittura della memoria può avere diversi bersagli, con conseguenze di gravità crescente:

- **Dati dell'utente:** altera i risultati del programma, ma non influisce su altri software in esecuzione.
- **Codice dell'utente:** la macchina proverà a eseguire un'istruzione non valida causando un crash, oppure, se il codice coincide con istruzioni malevole, le eseguirà.
- **Dati o codice di sistema:** ha gravissime implicazioni portando alla *Privilege Escalation* (scalata dei privilegi). Il Sistema Operativo invoca un programma utente aspettandosi di riprenderne il controllo alla chiusura; sovrascrivendo la memoria in questo frangente, l'attaccante forza l'OS a eseguire il proprio codice ottenendo uno status privilegiato.

Contromisure al Buffer Overflow

Poiché gli overflow dipendono da difetti di programmazione, la responsabilità di restare "entro i limiti" è condivisa tra programmatore, sistema operativo, compilatore e hardware. Le buone pratiche impongono di controllare sempre le lunghezze dei dati prima della scrittura, confermare che gli indici degli array siano validi, controllare il codice per errori di confine (*off-by-one*) e monitorare l'input per accettare solo i caratteri gestibili. È inoltre essenziale limitare i privilegi dei programmi in modo che un'eventuale compromissione non garantisca privilegi di sistema all'attaccante. A livello pratico, è molto utile l'uso di linguaggi di programmazione moderni (come Java, .NET o Python) che generano codice per verificare automaticamente i limiti, a differenza di linguaggi a basso livello come C e C++.

Una potente contromisura difensiva a livello strutturale è lo **Stack Canary** (Canarino). Questo meccanismo consiste nell'avvolgere ogni blocco dello stack (*stack frame*) con un livello protettivo, inserendo un valore di rilevamento manomissioni subito prima del *program counter*. Poiché l'attaccante deve sovrascrivere l'intera porzione di memoria per arrivare a reindirizzare il programma, finirà inevitabilmente per sovrascrivere e corrompere anche il Canary. Il codice di terminazione verifica se questo valore è cambiato: in caso affermativo, l'esecuzione viene interrotta neutralizzando l'attacco. Affinché funzioni, è cruciale che solo il Sistema Operativo (il difensore) possa generare e verificare tale valore.

Incomplete Mediation

La mediazione (*Mediation*) è il processo di controllo volto a confermare l'autorizzazione di un attore prima che un programma esegua l'azione richiesta su un oggetto. Se questo processo presenta un difetto, si parla di **Incomplete Mediation**, ovvero un grave fallimento nei controlli di accesso.

Questo accade spesso nelle applicazioni web quando il programmatore lascia dati sensibili (come il prezzo di un articolo) in campi esposti e modificabili dall'utente, ad esempio all'interno dell'URL. Se un utente malintenzionato altera questi valori e il sistema di backend non ne verifica l'autenticità prima dell'uso, il programma elaborerà la transazione con dati falsificati.

La soluzione a questa vulnerabilità è la *Complete Mediation* (Mediazione Completa), che deve essere implementata secondo i rigorosi concetti del *Reference Monitor*. Affinché la validazione sia efficace e sicura, questo monitor deve possedere tre proprietà fondamentali: deve essere semplice e di piccole dimensioni (per garantirne la correttezza), deve essere inaggirabile (*unbypassable*) e deve essere sempre invocato ad ogni richiesta senza eccezioni.

Esempio 1: Parametri esposti in un URL

Il primo esempio mostra un tipico URL per l'inserimento di dati:

`http://www.somesite.com/subpage/userinput.asp?parm1=(808)5551212&parm2=2019Jan17`

- **La dinamica:** In questo URL, le variabili `parm1` (che sembra un numero di telefono) e `parm2` (una data) passano dei dati al programma `userinput.asp`.
- **Il problema:** Cosa succede se l'utente inserisce dati diversi da quelli che il programma si aspetta? Un programmatore potrebbe pensare di aver risolto il problema costruendo un form web che valida l'input lato client (ovvero sul browser dell'utente).
- **La vulnerabilità:** L'applicazione rimane comunque vulnerabile perché il programmatore ha lasciato questi dati sensibili in una posizione (l'URL) a cui l'attaccante ha libero accesso e che può modificare manualmente. I dati si trovano in una condizione esposta e non controllata in modo sicuro.

Esempio 2: L'Attacco a un sito di E-commerce

Le slide approfondiscono il concetto con un caso "reale" molto più pericoloso, riguardante un carrello degli acquisti. Immagina la stringa generata per processare un ordine:

`http://www.things.com/order.asp?custID=101&part=555A&qy=20&price=10&ship=boat&shipcost=5&total=205`

- **La logica di base:** Il sistema calcola il totale con una formula molto semplice: $Total = qy * price + shipcost$. Inserendo i numeri presenti nell'URL otteniamo la quantità (20) moltiplicata per il prezzo (10\$) più il costo di spedizione (5\$), per un totale corretto di 205\$.
- **La Falla:** Il calcolo del totale e la validazione avvengono sul "frontend" (la parte del sito sul computer dell'utente) e non vengono ricontrollati in modo indipendente; per questo la mediazione è "incompleta".
- **L'Attacco:** Un utente malintenzionato può ignorare il form della pagina web e alterare direttamente i valori nell'URL prima di premere invio. Ad esempio, può abbassare arbitrariamente il prezzo a 1\$ e ricalcolare il totale a 25\$, inviando questa richiesta:
`http://www.things.com/order.asp?custID=101&part=555A&qy=20&price=1&ship=boat&shipcost=5&total=25`

Se il server riceve questa stringa e si fida dei parametri senza ricalcolare e mediare la transazione basandosi sui prezzi reali nel proprio database, l'attaccante riuscirà ad acquistare la merce a un prezzo stracciato (25\$ invece di 205\$).

Time-of-Check to Time-of-Use

Per migliorare l'efficienza, i moderni processori e i Sistemi Operativi spesso modificano l'ordine in cui vengono eseguite le procedure e le istruzioni. Questa dinamica temporale introduce la vulnerabilità **Time-of-Check to Time-of-Use (TOCTTOU)**.

Questa falla sfrutta specificamente la finestra temporale che intercorre tra il momento in cui il sistema controlla un'autorizzazione (*check*) e il momento in cui utilizza effettivamente il dato approvato (*use*). Un attaccante può approfittare di questo breve ritardo per sostituire un file autorizzato con uno malevolo, attuando una vera e propria truffa del tipo "esca e scambio" (*bait and switch*).

Questo sfruttamento comporta un fallimento diretto del controllo degli accessi, portando a compromissioni di riservatezza e integrità. Per difendersi, il software deve "possedere" i dati di richiesta dal momento del controllo fino al completamento dell'azione, garantendo l'integrità seriale e impedendo qualsiasi interruzione o cambio di stato durante la convalida. Una pratica standard consiste nel copiare i dati dallo spazio dell'utente a un'area sicura della routine di controllo accessi (totalmente fuori dalla portata dell'utente) ed eseguire le validazioni e le operazioni esclusivamente su questa copia protetta.

Il Primo Esempio: La Cassa del Negozio

Immagina di essere in un negozio. Vai alla cassa e metti sul bancone 5 banconote da 20 dollari per pagare un conto totale di 100 dollari. Il cassiere guarda i soldi e verifica che l'importo sia corretto (questo è il *Time-of-Check*). Subito dopo, però, il cassiere viene distratto da un altro cliente e si gira un attimo.

Approfittando di questa distrazione, tu riprendi indietro una banconota da 20 dollari. Quando il cassiere si volta nuovamente, prende il denaro rimasto sul bancone (ormai solo 80 dollari) e ti consegna la ricevuta e la borsa con gli acquisti (questo è il *Time-of-Use*). Tra il momento del controllo e il completamento della transazione è successo qualcosa di illecito perché la situazione non è stata bloccata e preservata.

Il Secondo Esempio: L'Accesso ai File

Supponiamo che una richiesta di accesso a un file venga presentata al sistema sotto forma di una struttura dati, la quale contiene il nome del file e la modalità di accesso richiesta.

Il processo che gestisce il Controllo degli Accessi (AC) riceve questa struttura dati, la analizza e stabilisce che tu hai i permessi necessari, approvando la richiesta. A questo punto, il sistema inoltra la struttura dati al gestore dei file (*file handler*) per eseguire l'operazione, ma questo passaggio richiede l'esecuzione di molte istruzioni e, di conseguenza, del tempo.

In questa frazione di secondo (il ritardo tra il controllo e l'uso), l'attaccante interviene sulla struttura dati originale. Come illustrato nei diagrammi delle slide, l'attaccante invia inizialmente una richiesta legittima: "*File: my_file, Azione: Cambia il byte 4 in A*". Il sistema la approva. Un attimo prima che l'azione venga eseguita, l'attaccante modifica la struttura dati (che è ancora nello spazio accessibile all'utente) in: "*File: your_file, Azione: Elimina il file*". Il gestore dei file riceve questa seconda istruzione e la esegue senza rifare i controlli, fidandosi dell'approvazione precedente.

Backdoor e Altri Errori di Programmazione

Durante le fasi di sviluppo e testing, i programmatori creano spesso dei punti di accesso non documentati per testare più agilmente il codice. Il problema di sicurezza emerge quando si dimenticano di rimuovere queste scorciatoie prima di rilasciare il software definitivo. Questi accessi nascosti prendono il nome di **Backdoor** (o Trapdoor) e permettono a chiunque li conosca di ottenere il controllo del sistema aggirando le normali procedure e acquisendo i privilegi preimpostati dal programmatore originario.

Le backdoor sono difficilissime da individuare tramite le revisioni del codice poiché non figurano in nessuna documentazione. Spesso, inoltre, vengono introdotte inavvertitamente durante le fasi di manutenzione da addetti che non possiedono specifiche formazioni nel campo della sicurezza informatica.

Oltre alle backdoor, esistono altre tipologie di sviste di programmazione comuni che possono causare malfunzionamenti o aprire la strada ad attacchi:

- **Off-by-One Error:** errori legati allo scarto di un'unità nel calcolo dei confini, classici nei cicli iterativi.
- **Integer Overflow:** il superamento dei limiti di memoria previsti per una variabile di tipo numerico intero.
- **Unterminated Null-Terminated String:** la mancanza del corretto carattere nullo necessario per chiudere e definire la fine di una stringa di testo.
- **Errori sui Parametri:** l'accettazione passiva di un numero eccessivo di parametri, di lunghezze non previste o di formati e tipi di dati errati.

Race Condition

La **Race Condition** è una problematica che si presenta quando il comportamento e l'esito finale di un programma dipendono dall'ordine imprevedibile in cui due procedure o processi indipendenti vengono eseguiti.

Questo fenomeno si verifica con maggior frequenza nei Sistemi Operativi complessi o nei processi multi-thread cooperanti. Per fare un esempio classico: in un sistema di prenotazione, se due processi verificano contemporaneamente la disponibilità di un medesimo posto libero, entrambi riceveranno esito positivo e procederanno alla prenotazione dello stesso posto, creando una sovrascrittura e corrompendo i dati.

Questa vulnerabilità è estremamente ardua da rilevare o replicare volontariamente perché si basa su variabili sistemiche labili. Tempistiche e ordine di esecuzione vengono infatti influenzati da elementi variabili come il carico totale sul sistema in quel preciso istante, la memoria disponibile, la priorità assegnata ai processi dal processore e le interruzioni di sistema (*interrupts*). Conseguentemente, le Race Condition causano esiti incoerenti e indesiderati, configurandosi come un chiaro e totale fallimento dell'integrità del sistema.

Riepilogo Impatti sulla Sicurezza (Triade CIA)

Ogni vulnerabilità descritta ha un peso specifico diverso rispetto ai pilastri della Sicurezza delle Informazioni (Riservatezza, Integrità, Disponibilità). La seguente tabella riassume quali proprietà vengono tipicamente compromesse in caso di attacco riuscito:

Tipo di Difetto (Flaw)	Proprietà CIA Compromessa
Incomplete Mediation	Integrità, Disponibilità
Buffer Overflow	Tutte (Riservatezza, Integrità, Disponibilità)
Time-of-check to time-of-use	Riservatezza, Integrità
Backdoor	Tutte (Riservatezza, Integrità, Disponibilità)
Race Condition	Integrità